

DMEI: Dynamic Memory Encryption and Indirection

A Moving-Target Dispatch Layer for Hardware-Register-Bound Secrets

Eyal Braun — Independent Security Research — 2026

Keywords: dispatch table security, memory encryption, moving-target defence, control-flow integrity, ROP mitigation, hardware secrets, AES-NI

Abstract

Dispatch tables—OS syscall vectors, interpreter opcode arrays, and real-time task routers—are among the most security-critical data structures in modern software. Their static layout makes them permanently exploitable: a single memory read defeats KASLR, Rowhammer flips redirect control flow silently, and code-page tampering subverts dispatch targets without touching the table itself. All existing defences are one-time initialisations.

We present DMEI (Dynamic Memory Encryption and Indirection), a defence-in-depth dispatch layer that prevents address resolution without simultaneously holding all hardware-resident secrets. DMEI combines two-hop AES-encrypted indirection keyed by hardware registers, a dual-trigger moving-target reshuffle, per-lookup function-body integrity via keyed MAC, an all-canary table with epoch-rotating position-dependent canaries, caller IP validation per dispatch identifier, TPM/TEE hardware key binding, hardware watchpoint spill detection, and encrypted return-address stack binding to defeat ROP. All lookups are $O(1)$.

I. Introduction

Function dispatch tables appear at every software layer: the Linux `sys_call_table`, CPython opcode array, and RTOS task schedulers all concentrate function pointers in a single structure. Corrupting even one entry redirects all future calls to attacker-controlled code.

A. The Problem: Static Tables Are Permanently Exploitable

- **Memory scanning.** A compromised process builds a complete identifier-to-address map with no write access required. KASLR is defeated by a single read.
- **Rowhammer / DMA corruption.** Bit-flips in table entries silently redirect dispatch without triggering any memory-protection mechanism.
- **Code-page tampering.** The attacker modifies the function body; the table resolves correctly but dispatches to modified instructions. Standard table MACs do not detect this.

B. Why Existing Solutions Fall Short

KASLR is defeated by a single memory read. WxOR prevents injection but leaves pointers readable. CFI and Intel CET restrict call targets but require the table to be readable in plaintext. ARM PAC authenticates pointers in transit but not table entries at rest. Shadow Call Stack protects return addresses but not forward dispatch. The common deficiency is temporal: every existing defence is a one-time initialisation.

C. Contributions

DMEI addresses the temporal dimension through seven mechanisms:

1. Two-hop AES-encrypted indirection keyed by hardware registers
2. Dual-trigger moving-target reshuffle ($J_time + J_count$)

3. Per-lookup function-body integrity via double-encrypted keyed MAC binding code content and canonical address
4. All-canary table with epoch-rotating position-dependent canaries — 100% detection of unregistered slot access
5. Caller IP validation with Enc_E(IP) equality check per dispatch identifier S
6. Hardware watchpoint spill detection
7. Encrypted return-address stack binding (ROP defence)

II. Threat Model

We give a precise statement of what the adversary can and cannot do, because every subsequent security claim depends on it.

A. Attacker Capabilities

- **Arbitrary RAM read/write.** The attacker can read any byte of physical memory and write to any byte not protected by hardware page permissions. This subsumes KASLR bypass, cold-boot attacks, and DMA attacks.
- **Rowhammer.** The attacker can induce bit-flips in DRAM rows adjacent to rows they access repeatedly, including in the dispatch table itself.
- **Cache-timing observation.** The attacker can mount Flush+Reload or Prime+Probe attacks. Specifically, the slot index accessed during a lookup is considered observable.
- **Oracle queries.** The attacker can trigger dispatch lookups from a registered call site and observe whether they succeed or trigger a reshuffle. They are rate-limited to J_limit queries per epoch.
- **Trigger reshuffles.** The attacker can force reshuffle events by flooding J_count or inducing canary trips. This is a DoS risk, not a confidentiality break: each reshuffle generates fresh secrets.

B. Attacker Limitations

- **No CPU register read.** The attacker cannot directly read CPU architectural registers (RAX, YMM0–YMM7). All secrets reside exclusively in the register file.
- **No microarchitectural register leak.** We explicitly assume the platform is not subject to register-value-leaking speculative execution attacks (e.g., Spectre v3a). On affected platforms, serialising instructions (LFENCE/MFENCE) should be added at lookup entry and exit.
- **Cannot break AES.** AES (128-bit key) is assumed computationally secure.
- **Cannot subvert TPM or TEE.** Hardware key binding via TPM PCR quotes and TEE enclaves is assumed correct. Physical platform attacks are out of scope.
- **Ring 0 (kernel) is trusted.** DMEI requires a trusted entity at Ring 0 for key generation and to authorise dynamic caller registration. A fully compromised kernel is outside the threat model. A partially compromised kernel — one that can write RAM but not directly read registers — remains in scope.

C. Reshuffle-Forcing as a DoS Vector

An attacker who can trigger arbitrary reshuffles by flooding lookups can cause repeated reshuffle overhead. This is a denial-of-service risk, not a confidentiality risk: each reshuffle generates fresh

cryptographic secrets, so an attacker gains nothing from forcing them. Deployments should set J_time and J_limit conservatively and rate-limit anomaly-triggering identifiers.

III. Core Idea and Goals

Goal. Prevent any attacker who can read or tamper with RAM from resolving or redirecting dispatch targets unless they simultaneously hold all ephemeral hardware-resident secrets.

Key strategy. Store only encrypted offsets in RAM. Keep all secrets (K, T, E, B, D_h, D_g, D_c) in CPU registers. Interpret the table only at lookup time.

Moving target. Periodically — or on any anomaly — reshuffle and re-derive all secrets so a RAM snapshot becomes obsolete within the current epoch (J_time).

Data flow invariant. Secrets are generated in registers and never written to RAM. Non-secret values move RAM→registers for computation; results move back to RAM. Secrets never flow register→RAM.

IV. Secrets and Data Structures

A. Register Vault

All secrets reside exclusively in CPU vector registers (YMM0–YMM7 on x86-64). They are derived from entropy XOR TPM PCR quote inside the register file and are never written to RAM in plaintext.

Secret	Lives In	Purpose
K	YMM register	Master key — input to all hash and MAC functions
T	YMM register	Global anchor — all offsets measured from T
E	YMM register	Slot encryption key for Enc_E / Dec_E
B	YMM register	Boundary index separating N and M sections
D_h	YMM register	Secret random domain separator for H
D_g	YMM register	Secret random domain separator for G
D_c	YMM register	Secret random domain separator for C

Table I — Register Vault: all secrets and their roles

B. Table Structure

The table is divided into two sections with a secret boundary at index B. The entire table initialises as canaries. Slots are punched open only for registered entries and caller IPs. Any access to an unregistered slot triggers an immediate reshuffle — 100% canary coverage with no probabilistic approximation.

N Section [0, B)	M Section [B, TABLE_SIZE)
Encrypted entry slots: I_j offset from T+T ₂ , T ₂ offsets, FMAC values, arm values. All other slots: CANARY	Encrypted caller IP slots: one slot per (IP, S) pair storing Enc_E(IP). All other slots: CANARY

Figure 1 — Table layout: N section (entries) and M section (caller IPs), boundary B secret

C. An Entry Structure

Each registered identifier S has a corresponding An entry storing three encrypted values at indexes derived from $H(S, D_h, K)$:

- **An₁ — arm:** A randomly generated value used as input to G , ensuring G can only be used with the correct arm value for that specific S .
- **An₂ — T₂:** A per-entry random offset. The actual function address I_j is $T + T_2 + \text{Dec}_E(\text{table}[j])$. T_2 is per-entry so offset values cannot be correlated across entries.
- **An₃ — FMAC:** $\text{Enc}_E(\text{AES}(\text{first_64_bytes}(I_j) \text{ XOR } \text{address}(I_j), K))$. The table entry is double-encrypted: the outer Enc_E layer requires E to read; the inner AES layer is keyed by K , making it unforgeable without K . A code modification fails on the content dimension; an address redirect fails on the address dimension.

The first 64 bytes of the function body are chosen as they typically include the function prologue and initial control-flow logic. This parameter is tunable: higher coverage (e.g., 128 bytes or full-function hashing) increases security at the cost of additional computation.

D. Per-Slot Integrity

All integrity guarantees are carried by per-slot mechanisms. There is no table-wide MAC. The table contains six distinct slot types; each has an independent integrity argument.

Canary slots (N and M sections). Defined as $\text{canary}[i] = K \text{ XOR } E \text{ XOR } i$. Epoch-rotating (K and E regenerate every reshuffle), position-unique (each index holds a distinct value), and indistinguishable from registered slots without E . A flip at an unmapped slot is unreachable by any registered S without knowledge of K, D_h, D_g, D_c — and therefore carries no exploitable consequence. Detected exactly when it becomes dangerous, at access time: $\text{table}[i] \text{ XOR } E \text{ XOR } i == K$ means canary, abort.

An₁ — arm slot. Stores $\text{Enc}_E(\text{arm})$. A flip causes Dec_E to produce a wrong arm value. $G(S, \text{wrong_arm}, K, D_g)$ then derives a wrong jump index j' which either lands on a canary (abort) or decrypts to a garbage offset producing a wrong I_j . FMAC then fails on both the content and address dimensions. Protected by arm-gating of G .

An₂ — T₂ slot. Stores $\text{Enc}_E(T_2)$. A flip corrupts T_2 , shifting the reconstructed address to $T + \text{corrupted_}T_2 + \text{offset}$. $\text{FMAC_now} = \text{AES}(\text{first_64_bytes}(\text{wrong_addr}) \text{ XOR } \text{wrong_addr}, K)$ will not match the stored FMAC. Protected by the address dimension of FMAC.

An₃ — FMAC slot. Stores $\text{Enc}_E(\text{AES}(\text{first_64_bytes}(I_j) \text{ XOR } \text{address}(I_j), K))$. To forge a passing value the attacker must simultaneously hold E (to produce a valid outer encryption) and K (to produce a valid inner AES binding). Both live exclusively in registers and are never written to RAM. Computationally unbreakable under the threat model.

table[j] — offset slot. Stores $\text{Enc}_E(I_j - T - T_2)$. A flip causes Dec_E to produce a garbage offset, reconstructing a wrong I_j . FMAC fails on both content and address dimensions before any dispatch occurs.

M-section — caller IP slots. Stores $\text{Enc}_E(\text{IP})$. The check at lookup time is: if $\text{table}[\text{index}] \neq \text{Enc}_E(\text{IP})$, abort. This is recomputed fresh each call using the live IP from $\text{return_address}()$ and E from the register file. A Rowhammer flip produces one of two outcomes. Case 1: the flip lands exactly on $K \text{ XOR } E \text{ XOR } \text{index}$ — the slot appears as a canary, the legitimate caller is rejected and a reshuffle fires. The attacker

cannot target this outcome without knowing K and E. Case 2: the flip produces any other non-canary value — it still will not equal $\text{Enc}_E(\text{caller_IP})$ which requires E to compute. The $\text{Enc}_E(\text{IP})$ equality check handles both unregistered callers and Rowhammer flips with a single comparison. No separate canary check is needed for the M-section.

V. Core Functions

A. $H(S, D_h, K)$ — Entry Index Derivation

H derives three table indexes for the A_n entry of identifier S, using sub-domain suffixes to produce distinct, non-colliding indexes:

```
An1.index = AES((D_h ⊕ 0x01) ⊕ S, K) mod N
An2.index = AES((D_h ⊕ 0x02) ⊕ S, K) mod N
An3.index = AES((D_h ⊕ 0x03) ⊕ S, K) mod N
```

Collisions are resolved via cuckoo eviction (max 8 kicks). If eviction fails the build phase restarts with a fresh entropy draw. D_h is secret so an attacker cannot compute these indexes without recovering D_h and K.

B. $G(S, \text{arm}, K, D_g)$ — Jump Index Derivation

$j = \text{AES}((D_g \text{ XOR } \text{arm}) \text{ XOR } S, K) \text{ mod } N$. G requires arm — the value stored at A_{n1} — so the two-hop lookup cannot be shortcut. An attacker who learns j learns only an encrypted offset; without E they cannot decrypt it.

C. $C(\text{IP}, K, D_c, S)$ — Caller IP Index Derivation

$\text{index} = \text{AES}((D_c \text{ XOR } S) \text{ XOR } \text{IP} \text{ XOR } K, K) \text{ mod } M + B$. S is mixed into the hash input, providing per-function enforcement with no separate data structure. The check at lookup time is:

```
index = C(IP, K, D_c, S)
if table[index] != Enc_E(IP):
    trigger reshuffle; abort    // unregistered or tampered caller
```

IP is known from `return_address()` and E is register-resident, so $\text{Enc}_E(\text{IP})$ is recomputed fresh each call in a single AES-NI operation. This single comparison handles all cases: unregistered callers, Rowhammer flips, and canaries all fail it.

D. $\text{arm}(K)$ — Irreversible Mixing

$\text{arm}(S_i) = \text{AES}(\text{RDRAND} \text{ XOR } K, K)$. Used to produce the arm value stored at A_{n1} . RDRAND ensures per-entry, per-epoch uniqueness.

VI. Build Phase Algorithm

The build phase constructs a fully encrypted, sealed dispatch table from scratch inside the register file. No secret value touches RAM at any point during this process. The phase runs once at initialisation and

again in full at every reshuffle. It proceeds in five sequential steps: key generation, canary initialisation, caller IP registration, entry registration, and sealing with an atomic swap.

```
// — Step 1: Key generation in registers —————
entropy = RDRAND @ urandom @ PCR_quote
seed    = 0x444D45494B455953454544434F4E5354    // hardcoded public constant
K       = AES(entropy, seed)
T       = RDRAND                                // fresh, independent of entropy
E       = AES(K, 0x455F4445524956455F434F4E5354) // "E_DERIVE_CONST"
D_h     = AES(K, 0x44485F4445524956455F434F4E53) // "DH_DERIVE_CONST"
D_g     = AES(K, 0x44475F4445524956455F434F4E53) // "DG_DERIVE_CONST"
D_c     = AES(K, 0x44435F4445524956455F434F4E53) // "DC_DERIVE_CONST"
B_raw   = AES(K, 0x425F4445524956455F434F4E5354) // "B_DERIVE_CONST"
B       = (B_raw mod (TABLE_SIZE / 2)) + (TABLE_SIZE / 4)
        // constrains B to [TABLE_SIZE/4, TABLE_SIZE*3/4)
N       = B                                     // size of N section = [0, B)
M       = TABLE_SIZE - B // size of M section = [B, TABLE_SIZE)

// — Step 2: Canary initialisation —————
// Epoch-rotating, position-unique canary. Indistinguishable from
// encrypted slots without E. Rotates automatically on every reshuffle.
for i in [0, TABLE_SIZE):
    table[i] = K XOR E XOR i

// — Step 3: Punch M slots — caller IP registration —————
// For each authorised (IP, S) pair, C() derives the slot index in the
// M section. Enc_E(IP) overwrites the canary.
for each (IP, S):
    index = C(IP, K, D_c, S)
    table[index] = Enc_E(IP)

// — Step 4: Punch N slots — entry registration —————
// arm and T2 generated fresh per entry per epoch.
// FMAC = AES(inner, K) — K-bound inner value.
// table[An3] = Enc_E(FMAC) — E-encrypted outer layer.
// Together: double-encrypted, requiring E to read and K to forge.
for each (Si, Ij):
    arm = AES(RDRAND @ K, K)
    T2 = RDRAND
    FMAC = AES(first_64_bytes(Ij) @ address(Ij), K) // inner K-bound MAC
    {An1.index, An2.index, An3.index} = H(Si, D_h, K)
    j = G(Si, arm, K, D_g)
    table[An1.index] = Enc_E(arm)
    table[An2.index] = Enc_E(T2)
    table[An3.index] = Enc_E(FMAC) // outer E layer — entry is Enc_E(AES(...))
    table[j]         = Enc_E(Ij - T - T2)

// — Step 5: Seal & atomic swap —————
// No table-wide MAC. Integrity is enforced per-slot at lookup time.
atomic CMPXCHG on active_ptr → new table goes live for all subsequent lookups
explicit_bzero(old_table) // forensic burn after in-flight lookups drain
reset J_c = 0
re-randomise J_time // drawn from [T_min, T_max]
```

VII. Lookup Phase Algorithm

The lookup phase resolves a dispatch identifier *S* from a registered call site to its target function, verifies all integrity properties, and executes the dispatch — all without exposing any secret to RAM. It proceeds in six sequential steps.

```

// — Step 1: Validate caller IP —————
// Recompute Enc_E(IP) from the live return address and register-resident E.
// Any unregistered caller, Rowhammer flip, or canary fails this check.
IP = return_address()
index = C(IP, K, D_c, S)
if table[index] != Enc_E(IP):
    trigger reshuffle; abort // unregistered or tampered

// — Step 2: Resolve An entry (first hop) —————
// H derives the three An slot indexes for S. A canary at any slot means
// this identifier was never registered — abort immediately.
{An1.index, An2.index, An3.index} = H(S, D_h, K)
for each An_k.index:
    if table[An_k.index] XOR E XOR An_k.index == K:
        trigger reshuffle; abort // unregistered identifier
arm = Dec_E(table[An1.index], E)
T2 = Dec_E(table[An2.index], E)
FMAC = Dec_E(table[An3.index], E) // recovers inner AES(first_64... , K)

// — Step 3: Resolve jump slot (second hop) —————
// G requires arm, so the two-hop cannot be shortcut.
j = G(S, arm, K, D_g)
if table[j] XOR E XOR j == K:
    trigger reshuffle; abort // slot was never punched
offset = Dec_E(table[j], E)
I_j = T + T2 + offset // absolute address reconstructed in registers

// — Step 4: Verify function-body integrity —————
// Recompute inner K-bound MAC and compare to Dec_E result from An3.
// Binds both code content and canonical address simultaneously.
FMAC_now = AES(first_64_bytes(I_j) @ address(I_j), K)
if FMAC_now != FMAC:
    trigger reshuffle; abort // code-page tampering or address redirect

// — Step 5: Update jump counter and moving-target triggers —————
J_c = J_c + 1
if J_c >= J_limit → schedule reshuffle // J_count trigger
if now >= J_time → schedule reshuffle // J_time trigger

// — Step 6: Dispatch —————
goto *I_j

```

VIII. Caller IP Validation

A. Design

Caller IP validation enforces that every DMEI function can only be invoked from code authorised at build time or explicitly registered by a trusted party. *S* is used as the per-function domain input to *C*, requiring no separate function ID concept and no additional data structure. The check is a single $\text{Enc}_E(\text{IP})$ equality comparison — $O(1)$, constant-time, no branch on secret.

B. Static Registration — Build-Time Binary Scan

The initial set of valid (IP, *S*) pairs is extracted automatically from the compiled binary:

8. Disassemble the binary and locate every call instruction whose target is a DMEI function.
9. Record the return address of each call (the instruction immediately after the call).

10. For each (return_address, S) pair, compute $C(\text{return_address}, K, D_c, S)$ and punch that slot in the M section with $\text{Enc_E}(\text{return_address})$.

ASLR is handled by storing offsets relative to the binary base and resolving to absolute addresses at init.

C. Dynamic Registration

New caller IPs are registered via `dmei_add_caller(IP, token, S, caller_ip)`. `token = arm(IP, K)` must be computed by a party already holding `K` (Ring 0 trusted party). Verification: `ct_eq64(arm(IP, K), token)` — constant-time, `K` never in a branch condition, no cache exposure. The caller of `dmei_add_caller` must itself be a registered caller for `S_addcaller`, preventing privilege escalation.

IX. Asynchronous Reshuffle Model

A. Concurrency Contract

A reshuffle must not stall in-flight lookups, must not corrupt them, and must not leave any epoch's secrets observable after the swap. The protocol:

11. **Background build.** A dedicated reshuffle thread allocates a new table buffer and generates fresh secrets entirely in registers. The old table remains live and fully functional for concurrent lookups.
12. **Seal.** Integrity of the new table is enforced per-slot at each subsequent lookup via the `Enc_E(IP)` and FMAC checks. No table-wide MAC pass is required.
13. **Atomic pointer swap.** A single `CMPXCHG` on `active_ptr` switches all subsequent lookups to the new table atomically. Lookups already in flight hold a hazard pointer to the old table and complete safely.
14. **Forensic burn.** Once all in-flight lookups on the old epoch have drained (epoch reference count reaches zero), `explicit_bzero` wipes the old table and all old secrets. Cache lines containing old epoch data remain but are cryptographically inert: all slots are ciphertext under a dead key that no longer exists anywhere.

B. Formal Guarantees

- **No mixed-epoch lookups.** A lookup always reads from a single epoch's table; it never straddles a swap.
- **No stall on reshuffle.** Lookups running against the old epoch continue uninterrupted during the background rebuild.
- **No epoch leakage after burn.** Once the forensic burn completes, no plaintext secret of the old epoch exists in RAM.

C. Dual-Trigger Conditions

Reshuffle fires under any of: `J_time` elapsed; `J_c` $\geq$ `J_limit`; canary trip; spill detection; rate breach; or unregistered caller. An atomic flag de-duplicates concurrent triggers so exactly one reshuffle executes per anomaly event.

X. Spill Security

A. The Spill Problem

Under register pressure the compiler may spill a register value to the stack frame. If K, E, or any secret is spilled, it briefly exists in RAM and is visible to an attacker with a RAM dump. Standard register keyword hints in C are not guaranteed to prevent spilling.

B. Spill Location Formula

$\text{SpillBase} = \text{RBP} - \text{roundTo16}((\text{RBP} - \text{RSP}) + \text{SizeOfRegisters})$, where $\text{SizeOfRegisters} = 32 \times (\text{number of protected YMM secrets})$. A single DR0 hardware watchpoint placed on SpillBase fires a #DB exception before any secret lands in RAM, zeroing all registers and triggering a reshuffle. The attacker gains a detection event, not a secret value. The watchpoint address is recomputed on each function entry.

C. Assumptions and Portability

This mechanism assumes a standard x86-64 ABI stack frame layout. Aggressive compiler optimisations or non-standard calling conventions may alter spill placement. In such environments, widening the monitored region or using multiple watchpoints may be required to preserve detection guarantees.

XI. Encrypted Return-Address Binding (ROP Defence)

A. Motivation

Return-Oriented Programming (ROP) hijacks control flow without injecting code: the attacker overwrites the saved return address on the stack to chain existing gadgets and bypass W[⊗]X entirely. Standard stack canaries have two fundamental weaknesses: an attacker who can read RAM recovers the canary value and writes it back verbatim around the overwrite; and an attacker with a non-linear write primitive skips the canary entirely. DMEI's encrypted binding is immune to both since the encryption key E never appears in RAM.

B. Mechanism

Entry — push encrypted return address: Read $\text{ret} \leftarrow [\text{RSP}]$; compute $\text{enc_ret} = \text{Enc_E}(\text{ret})$ entirely in registers; push enc_ret onto the stack. The stack now holds both the live return address and its encrypted shadow.

Exit — verify encrypted return address: Pop saved_enc from the stack; re-read $\text{ret}' \leftarrow [\text{RSP}]$; recompute $\text{enc_ret}' = \text{Enc_E}(\text{ret}')$ in registers; constant-time compare $\text{ct_eq64}(\text{enc_ret}', \text{saved_enc})$. Mismatch $\Rightarrow$ trigger reshuffle + fault; the corrupted return address is never used.

C. Why This Defeats ROP

A ROP attacker overwrites $[\text{RSP}]$ with gadget address g . At exit, $\text{Enc_E}(g) \neq \text{Enc_E}(\text{ret})$ since AES is a bijection. The comparison fails, reshuffle fires, and ret is never executed. Forging a matching saved_enc requires computing $\text{Enc_E}(g)$ for the attacker's chosen g — which requires E, which lives only in a CPU register and is never written to RAM.

XII. Concrete Deployment: CPython Opcode Dispatch

A. Where S Comes From

In CPython's `ceval.c`, each bytecode instruction is dispatched via a computed goto or switch on the opcode byte. DMEI replaces this with `dmei_lookup(S = opcode, IP = return_address())`. `S = opcode` (0–255). Each opcode handler address is registered during interpreter init as `dmei_register(opcode, handler_fn_ptr)`.

B. How Caller IPs Are Discovered

The static binary scan disassembles python3 and locates every call `dmei_lookup` instruction. Each call site's return address is paired with the `S` value from the preceding `mov rdi, ...` instruction and registered in the `M` section at init. Because CPython's eval loop is a single hot function, there are typically one or two call sites. Dynamic call sites (from JIT-compiled code or ctypes callbacks) are registered via `dmei_add_caller` at module load time.

C. Triggering Reshuffles in Practice

- **J_time:** Drawn uniformly from $[T_{\min}, T_{\max}]$ each epoch. Recommended ranges: [300 ms, 700 ms] for a REPL session; [30 ms, 80 ms] for a web server handling untrusted bytecode. $T_{\max}$ upper-limits the worst-case epoch duration an attacker can synchronise to. Randomisation prevents epoch-boundary synchronisation attacks.
- **J_limit:** Set to 10^6 lookups (roughly 100k Python bytecodes at CPython's typical dispatch rate).
- **Anomaly triggers:** Any canary trip in the opcode table (e.g., from a buffer overflow touching `opcode_targets`) fires an immediate reshuffle and alert.

D. Security Benefit in Context

An attacker who exploits a CPython memory disclosure sees only encrypted offsets under unknown `E`. An attacker who flips bits via Rowhammer causes an `Enc_E(IP)` or FMAC mismatch on the next dispatch — the corrupted entry is never executed. An ROP attacker who overwrites the eval loop's return address triggers the encrypted-binding check and is stopped before the gadget chain starts.

XIII. Security Analysis

A. Against Memory Scanning

An attacker with full RAM read access sees only: epoch-rotating position-dependent canaries ($K \oplus E \oplus i$ — indistinguishable from encrypted slots without `E`), encrypted offsets under `E`, `An` values under `E`, and encrypted caller IPs under `E`. `B` is secret so the `N/M` boundary is unknown. `D_h`, `D_g`, `D_c` are secret so hash outputs cannot be predicted.

B. Against Rowhammer / DMA Corruption

A flip in any `N`-section registered slot (`arm`, `Tz`, `FMAC`, `offset`) is caught by the `arm/FMAC` chain before dispatch. A flip in an `N`-section canary slot at index `i` is detected at access time (`table[i]  $\oplus$  E  $\oplus$  i == K`) and is otherwise unreachable since slot indexes derive from secret `K`, `D_h`, `D_g`. A flip in any `M`-section slot is caught by the `Enc_E(IP)` equality check: the corrupted value will not equal `Enc_E(caller_IP)` recomputed from the live `IP` and register-resident `E`. Rowhammer on the code page is caught by `FMAC`, which binds the first 64 bytes of the function body.

C. Against Caller Forgery

Registering a new caller IP requires producing $\text{arm}(\text{IP}, K)$ without K . K lives only in registers and is never written to RAM. The only way to obtain K is Ring 0 access — at which point the entire threat model is already violated.

D. Against Oracle Enumeration

Each lookup increments J_c . After J_{limit} lookups all secrets rotate. An attacker attempting to enumerate valid S values via oracle queries is rate-limited to J_{limit} queries per epoch. Across epochs all secrets — including K , E , D_h , D_g , D_c — change, invalidating all prior observations.

E. Against ROP

Covered in Section XI. The encrypted return-address binding makes a tampered return address detectable before it is ever executed, without requiring a separate memory region or hardware feature.

F. Ablation Analysis: Why Each Mechanism Is Necessary

- **A1: Remove caller IP validation.** An attacker who learns a valid S can call dmei_lookup from arbitrary injected or ROP code, enabling unrestricted oracle queries and brute-forcing G 's output within an epoch.
- **A2: Remove FMAC.** A precise Rowhammer flip targeting only the j -slot offset produces a silently incorrect I_j that dispatches to attacker-controlled code. FMAC is the mechanism closing this gap.
- **A3: Drop reshuffle on canary trips.** An attacker can probe indefinitely, mapping which slots are registered vs. canary in $O(\text{TABLE_SIZE})$ queries, undermining all index-hiding properties.
- **A4: Remove hardware watchpoint spill detection.** A compiler under register pressure silently spills K or E to the stack. A single RAM dump then reveals the epoch's master key. Watchpoints convert this silent failure into a detectable anomaly.
- **A5: Remove encrypted return-address binding.** A ROP attacker who corrupts the dispatch function's return address proceeds undetected — none of A1–A4 guard the stack directly.

G. Against Cache-Timing

K and E never appear in branch conditions. Residual leakage (slot index observable via Flush+Reload) is discussed in Section XIV.

XIV. Side-Channel Analysis

DMEI's register-only secret storage eliminates cryptographic key leakage via cache timing. An attacker mounting Flush+Reload during a lookup observes which table slot was accessed — a function of H , G , C , all keyed by unknown register secrets — and that a lookup occurred. Timing leaks the fact of dispatch but not the target.

An attacker who accumulates slot-index observations across multiple lookups within a single epoch gains a partial map of which slots are accessed for which observable S values. However, because D_h , D_g , and D_c rotate on every reshuffle, all slot-index observations become invalid at epoch boundary. The useful observation window is therefore bounded by $\min(J_{\text{limit}}, J_{\text{time}})$: an attacker who forces a reshuffle by flooding lookups destroys their own accumulated map. This creates an inherent tension for the attacker between collecting observations and triggering the rotation that invalidates them.

Deployments on platforms subject to Spectre v1 or v2 should insert LFENCE after the caller IP check and before any secret-dependent branch. DMEI's threat model explicitly excludes register-value-leaking

speculative execution (Spectre v3a); on affected platforms, serialising instructions at lookup entry and exit close this gap without altering the logical correctness of the protocol.

XV. Performance

DMEI achieves fully $O(1)$ lookup. Every step of the lookup phase is a fixed number of AES-NI operations plus constant memory reads, regardless of table size. The complete per-lookup cost is approximately 5–6 AES-NI round-trip operations: one for C (caller IP index), three for H (An slot indexes), one for G (jump slot index), and one for FMAC recomputation. On modern x86-64 microarchitectures, each AES-NI round executes in 3–4 cycles; the total lookup overhead is on the order of 20–40 cycles exclusive of memory latency.

The encrypted return-address binding invokes AES-NI at both function entry and exit, consuming approximately 50–100 cycles inclusive of the constant-time comparison and any branch serialisation. Applied uniformly to every function call in a general-purpose execution environment this would constitute an unacceptable regression. DMEI targets critical boundary transitions — the Linux `sys_call_table`, CPython's opcode dispatch array, and analogous high-value junctions — where this cost is structurally negligible: a Linux syscall crossing Ring 3–Ring 0 incurs 100–300 cycles in SYSCALL/SYSRET microcode alone.

XVI. Limitations and Future Work

DMEI's security guarantees are conditioned on the register file remaining unreadable to the attacker. Microarchitectural attacks that leak register values — including Spectre v3a and MDS-class vulnerabilities — are explicitly out of scope and represent the primary avenue for future hardening. Platforms with hardware support for register-file isolation (e.g., future TEE extensions) would allow the threat model to be tightened.

The `Enc_E(IP)` equality check in the M-section is evaluated fresh on every lookup from a registered call site, which for a single hot eval loop (e.g., CPython's dispatch) is one or two sites. For deployments with a very large number of distinct registered call sites, the per-lookup AES-NI cost of recomputing `Enc_E(IP)` is constant but may warrant pre-caching the encrypted value at the call site in a register across loop iterations.

Caller IP registration currently requires a static binary scan at init time. Dynamically generated code (JIT, self-modifying trampolines) must register call sites explicitly via `dmei_add_caller`. Extending the static scanner to handle common JIT patterns is left as future work.

XVII. Related Work

Defence	Continuous	Hides Addr.	Probe Det.	RAM-dump safe	Caller Auth.	ROP Def.
KASLR	—	~	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>
PointGuard	—	~	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>
CFI / CET	✓	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	~
ARM PAC	~	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	~
Shadow Stack	✓	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	✓

COOP	N/A	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>
DMEI (this work)	✓	✓	✓	✓	✓	✓

Table II — Comparison with existing dispatch protection schemes (✓ = yes, *x* = no, ~ = partial)

KASLR provides one-time address randomisation defeated by a single read. PointGuard XORs stored pointers with a random value — one-time obfuscation, no rotation. CFI and Intel CET restrict what an attacker can do after learning an address; DMEI prevents learning it. ARM PAC authenticates pointers in transit but not table entries at rest. Shadow Call Stack protects return addresses but not forward dispatch; DMEI's encrypted binding is a software complement that requires no hardware support. COOP chains virtual-dispatch entries — DMEI's encrypted table makes those entries unreadable, preventing gadget chain construction.

XVIII. Conclusion

DMEI demonstrates that dispatch tables can be made computationally opaque to an attacker with full RAM access, without sacrificing $O(1)$ lookup performance or real-time deployment viability. The core insight is temporal: by binding all table interpretability to ephemeral hardware-resident secrets and rotating those secrets on a bounded schedule, a RAM snapshot is never sufficient to resolve any dispatch target.

The explicit threat model clarifies that microarchitectural register leaks and fully compromised kernels are out of scope, while partial Ring 0 compromise — RAM write without register read — is precisely the class DMEI defends against. The asynchronous reshuffle model provides three formal guarantees: no mixed-epoch lookups, no stall on reshuffle, and no epoch leakage after forensic burn. The ablation analysis shows each of the seven mechanisms is independently necessary. The encrypted return-address binding extends DMEI's RAM-confidentiality guarantees to the control-flow stack, closing the ROP gap without requiring separate hardware or memory regions.

An attacker must simultaneously break AES, forge the double-encrypted FMAC, bypass Enc_E(IP) caller validation, avoid triggering any of six independent anomaly detectors, and forge the encrypted return-address binding — all within a single reshuffle epoch. No existing attack class achieves this.

References

- [1] H. Shacham et al., "On the effectiveness of address-space randomization," in Proc. CCS, 2004, pp. 298–307.
- [2] M. Abadi et al., "Control-flow integrity," in Proc. CCS, 2005, pp. 340–353.
- [3] ARM Ltd., Pointer Authentication in ARMv8.3, ARM Architecture Reference Manual, 2019.
- [4] R. Pagh and F. F. Rodler, "Cuckoo hashing," in Proc. ESA, 2001, pp. 121–133.
- [5] Y. Kim et al., "Flipping bits in memory without accessing them," in Proc. ISCA, 2014, pp. 361–372.
- [6] C. Cowan et al., "PointGuard: Protecting pointers from buffer overflow vulnerabilities," in Proc. USENIX Security, 2003, pp. 91–104.
- [7] S. Jajodia et al., Moving Target Defense. Springer, 2011.
- [8] Trusted Computing Group, TPM Main Specification, TCG, 2019.
- [9] Intel Corporation, Intel 64 and IA-32 Architectures Software Developer Manual, Vol. 3, 2023.
- [10] Intel Corporation, Control-flow Enforcement Technology Specification, 2020.
- [11] F. Schuster et al., "Counterfeit object-oriented programming," in Proc. IEEE S&P, 2015, pp. 745–762.

[12] P. Kocher et al., "Spectre attacks: Exploiting speculative execution," in Proc. IEEE S&P, 2019, pp. 1–19.